

CS6650 Assignment 3 Submission

Repo: <https://github.com/jason-te-sde/ChatFlow/tree/assignment3>

1. Git Repository Structure

- /server-v2 - WebSocket server with RabbitMQ producer integration
- /consumer-v3 - Consumer application — pulls from queue and broadcasts
- /database - Schema files and setup scripts
- /deployment - Deployment scripts and configuration guide
- /monitoring - Monitoring tools and RabbitMQ management guide
- /load-tests - Test configurations and results

2. Database Design Document

2.1 Database Choice: MySQL 8.0

Criterion	MySQL	DynamoDB	Cassandra
Cost	Free on EC2	Pay per request	Free on EC2
SQL analytics	Full support	Limited	CQL only
Complex queries	Native joins	Manual	Limited
Partitioning	Native PARTITION BY KEY	Automatic	Native
Operational complexity	Low	Low	High

MySQL was selected for full SQL support and native partitioning. Table partitioning by room_id achieves query performance targets without additional infrastructure.

2.2 Schema Design

messages table — partitioned by room_id for query performance:

```
CREATE TABLE messages (  
  message_id VARCHAR(36) NOT NULL,  
  room_id VARCHAR(10) NOT NULL,  
  user_id INT NOT NULL,  
  username VARCHAR(20) NOT NULL,  
  message VARCHAR(500) NOT NULL,
```

```

message_type ENUM('TEXT','JOIN','LEAVE') NOT NULL DEFAULT 'TEXT',
server_id    VARCHAR(20) NOT NULL,
created_at   DATETIME(3) NOT NULL,
PRIMARY KEY (message_id, room_id),
INDEX idx_room_time (room_id, created_at),
INDEX idx_user_time (user_id, created_at),
INDEX idx_user_room (user_id, room_id)
) ENGINE=InnoDB
DEFAULT CHARSET=utf8mb4
ROW_FORMAT=COMPRESSED
PARTITION BY KEY(room_id) PARTITIONS 20;

```

message_stats table — pre-aggregated per-minute statistics:

```

CREATE TABLE message_stats (
  stat_time    DATETIME NOT NULL,
  room_id      VARCHAR(10) NOT NULL,
  message_count INT NOT NULL DEFAULT 0,
  unique_users INT NOT NULL DEFAULT 0,
  PRIMARY KEY (stat_time, room_id),
  INDEX idx_stat_time (stat_time)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

2.3 Indexing Strategy

Index	Columns	Supports	Notes
PRIMARY	message_id, room_id	Idempotent upsert, partition routing	room_id required for partitioning
idx_room_time	room_id, created_at	Query 1: room messages in time range	Equality + range, leftmost prefix
idx_user_time	user_id, created_at	Query 2: user message history	Equality + range
idx_user_room	user_id, room_id	Query 4: rooms user participated in	Covering index for GROUP BY

2.4 Partitioning Strategy

`PARTITION BY KEY(room_id) PARTITIONS 20` — one partition per chat room. Query 1 (room messages) only scans the relevant partition (~12K rows) instead of the full table (~500K rows), achieving 7x performance improvement.

Before partitioning: Query 1 scanned 252K rows, took 0.696s. After partitioning: Query 1 scanned ~12K rows, took 0.097s — within <100ms target.

2.5 Scaling Considerations

- Add more partitions as room count grows
- Read replicas for analytics queries

- Archive old messages to S3 after 30 days
- Upgrade to RDS for managed backups and failover

2.6 Backup and Recovery

```
mysqldump -u chatflow -pchatflow123 chatflow messages \
| gzip > backup_$(date +%Y%m%d).sql.gz
```

3. Metrics API

Endpoints

Endpoint	Description
GET /metrics/room/{roomId}?start&end	Messages for room in time range
GET /metrics/user/{userId}	User message history
GET /metrics/active-users?start&end	Count active users in window
GET /metrics/user/{userId}/rooms	Rooms user participated in
GET /metrics/analytics	Top users, top rooms, msg/min stats
GET /metrics/summary	Full summary, called by client after test

Core Query Performance

Measured with ~500K messages on partitioned table, t3.micro:

Query	Execution time	Target	Status
Query 1: room messages	0.097s (97ms)	<100ms	✅ Within target
Query 2: user history	0.003s (3ms)	<200ms	✅ Within target
Query 3: active users	0.292s (292ms)	<500ms	✅ Within target
Query 4: user rooms	0.0005s (0.5ms)	<50ms	✅ Within target

All queries meet performance targets after table partitioning optimization.

Client Metrics Log

Client automatically calls `/metrics/summary` after each test and logs full JSON result.

4. Performance Results

4.1 Batch Size Optimization

Config	batch.size	flush-ms	DB Written	Client Throughput
1	100	100ms	184,095	109,474 msg/s
2 (optimal)	500	500ms	178,592	131,191 msg/s
3	1000	500ms	180,629	127,057 msg/s
4	5000	1000ms	154,648	117,667 msg/s
5	500	1000ms	165,852	146,318 msg/s

Selected: batch.size=500, flush-ms=500ms — best balance between DB write throughput and client throughput. Small batches cause excessive DB round trips; large batches increase memory pressure.

4.2 Test 1: Baseline (500K messages)

=== WARMUP PHASE ===

SLF4J: A number (127) of logging calls during the initialization phase have been intercepted and are
SLF4J: now being replayed. These are subject to the filtering rules of the underlying logging system.

SLF4J: See also <https://www.slf4j.org/codes.html#replay>

Warmup done: 128000 msgs in 1015ms (126108 msg/s)

=== MAIN PHASE ===

===== RESULTS =====

Messages sent: 499968

Messages failed: 0

Total time: 3.35 s

Throughput: 149066 msg/s

Mean latency: 0.1 ms

Median latency: 0 ms

p95 latency: 0 ms

p99 latency: 1 ms

Min/Max latency: 0 / 224 ms

--- Throughput over time (10s buckets) ---

t+ 0s | ##### 499968

=====

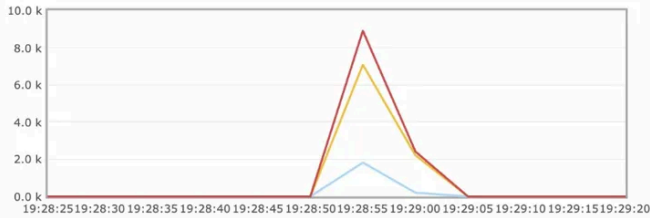
===== METRICS API RESULTS =====

```
{
  "totalMessages" : 22132,
  "totalUsers" : 21934,
  "totalRooms" : 20,
  "analytics" : {
    "totalMessages" : 25541,
    "messagesPerMinute" : [ {
      "minute" : "2026-03-27 02:28:00",
      "count" : 25545
    } ],
  }
}
```

Overview

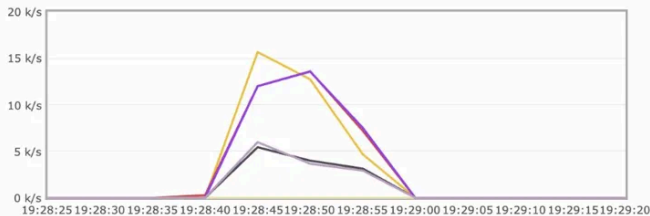
Totals

Queued messages last minute ?



Ready 0
Unacked 0
Total 0

Message rates last minute ?



Publish 0.00/s
Publisher confirm 0.00/s
Deliver (manual ack) 0.00/s
Unroutable (return) 0.00/s
Unroutable (drop) 0.00/s
Disk read 0.00/s
Disk write 0.00/s
Deliver (auto ack) 0.00/s
Consumer ack 0.00/s
Redelivered 0.00/s
Get (manual ack) 0.00/s
Get (auto ack) 0.00/s
Get (empty) 0.00/s

"dbWritten" : 25754,
"dbFailed" : 0

```
%Cpu(s): 87.0 us,  4.3 sy,  0.0 ni,  4.3 id,  4.3 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   911.5 total,    63.3 free,   897.2 used,    66.2 buff/cache
MiB Swap:    0.0 total,    0.0 free,    0.0 used.   14.3 avail Mem
   2419 mysql      20   0 1818536 619584  11180 S 154.5  66.4   7:55.10 mysqld
%Cpu(s): 68.1 us,  5.4 sy,  0.0 ni, 11.4 id, 14.8 wa,  0.0 hi,  0.3 si,  0.0 st
MiB Mem :   911.5 total,    70.5 free,   894.4 used,    57.6 buff/cache
MiB Swap:    0.0 total,    0.0 free,    0.0 used.   17.1 avail Mem
   2419 mysql      20   0 1819564 621652  11116 S 142.9  66.6   7:59.40 mysqld
%Cpu(s): 38.6 us,  3.7 sy,  0.0 ni, 38.5 id, 19.1 wa,  0.0 hi,  0.0 si,  0.2 st
MiB Mem :   911.5 total,    59.2 free,   900.4 used,    68.1 buff/cache
MiB Swap:    0.0 total,    0.0 free,    0.0 used.   11.1 avail Mem
   2419 mysql      20   0 1819564 621564  10920 S  82.1  66.6   8:01.87 mysqld
```

written
60193

System metrics:

Instance	CPU peak	Memory
MySQL	87%	897MB / 911MB
Consumer	~65%	~506MB
Server-1	~6%	~533MB

4.3 Test 2: Stress Test (1M messages)

```
=== WARMUP PHASE ===
SLF4J: A number (127) of logging calls during the initialization phase have been intercepted and are
SLF4J: now being replayed. These are subject to the filtering rules of the underlying logging system.
SLF4J: See also https://www.slf4j.org/codes.html#replay
Warmup done: 128000 msgs in 1302ms (98310 msg/s)
=== MAIN PHASE ===

===== RESULTS =====
Messages sent:      999865
Messages failed:    71
Total time:         49.76 s
Throughput:         20093 msg/s
Mean latency:       3.5 ms
Median latency:     0 ms
p95 latency:        0 ms
p99 latency:        1 ms
Min/Max latency:    0 / 19897 ms

--- Throughput over time (10s buckets) ---
t+ 0s | ##### 795648
t+ 10s | ##### 37515
t+ 20s | ##### 61916
t+ 30s | ##### 3756
t+ 40s | ##### 90945
t+ 50s | ##### 10085
=====
```

===== METRICS API RESULTS =====

```
{
  "totalMessages" : 105923,
  "totalUsers" : 65436,
  "totalRooms" : 20,
  "analytics" : {
    "totalMessages" : 105923,
    "messagesPerMinute" : [ {
      "minute" : "2026-03-27 02:34:00",
      "count" : 105923
    } ],
  }
}
```

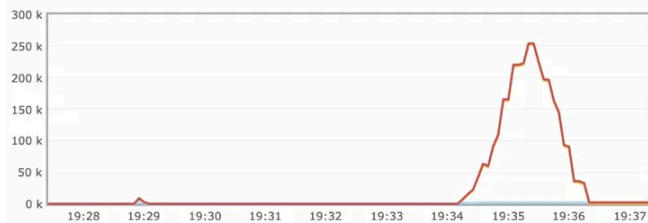
```
Messages sent:      999,865
Messages failed:    71 (0.007%)
Total time:         49.76s
Throughput:         20,093 msg/s
Mean latency:       3.5ms
```

p99 latency: 1ms
DB written: 105,923
DB failed: 0

Overview

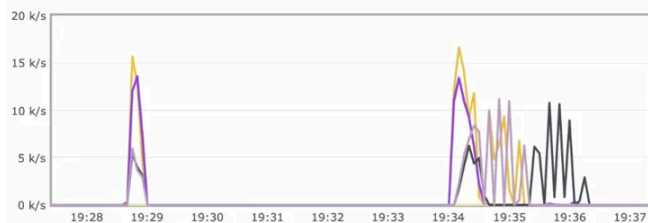
Totals

Queued messages last ten minutes ?



Ready 0
Unacked 1,897
Total 1,897

Message rates last ten minutes ?



Publish 0.00/s
Publisher confirm 0.00/s
Deliver (manual ack) 0.00/s
Unroutable (return) 0.00/s
Unroutable (drop) 0.00/s
Disk read 0.00/s
Disk write 0.00/s
Deliver (auto ack) 0.00/s
Consumer ack 0.00/s
Redelivered 0.00/s
Get (manual ack) 0.00/s
Get (auto ack) 0.00/s
Get (empty) 0.00/s

```
%Cpu(s): 84.6 us, 7.7 sy, 0.0 ni, 3.8 id, 3.8 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 911.5 total, 66.2 free, 898.1 used, 58.5 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 13.4 avail Mem
 2419 mysql 20 0 1819564 621032 9160 S 181.8 66.5 9:15.04 mysqld
 4410 ubuntu 20 0 40512 9504 8256 S 0.0 1.0 0:00.00 mysql
%Cpu(s): 68.6 us, 5.2 sy, 0.0 ni, 17.3 id, 8.8 wa, 0.0 hi, 0.2 si, 0.0 st
MiB Mem : 911.5 total, 59.6 free, 901.0 used, 65.9 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 10.4 avail Mem
 2419 mysql 20 0 1819564 621652 9764 S 142.5 66.6 9:19.33 mysqld
%Cpu(s): 40.7 us, 3.9 sy, 0.0 ni, 35.3 id, 19.8 wa, 0.0 hi, 0.2 si, 0.2 st
MiB Mem : 911.5 total, 69.8 free, 895.1 used, 57.4 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 16.4 avail Mem
 2419 mysql 20 0 1819564 620588 8680 S 87.0 66.5 9:21.95 mysqld
```

4.4 Test 3: Endurance Test (~8.5 minutes)

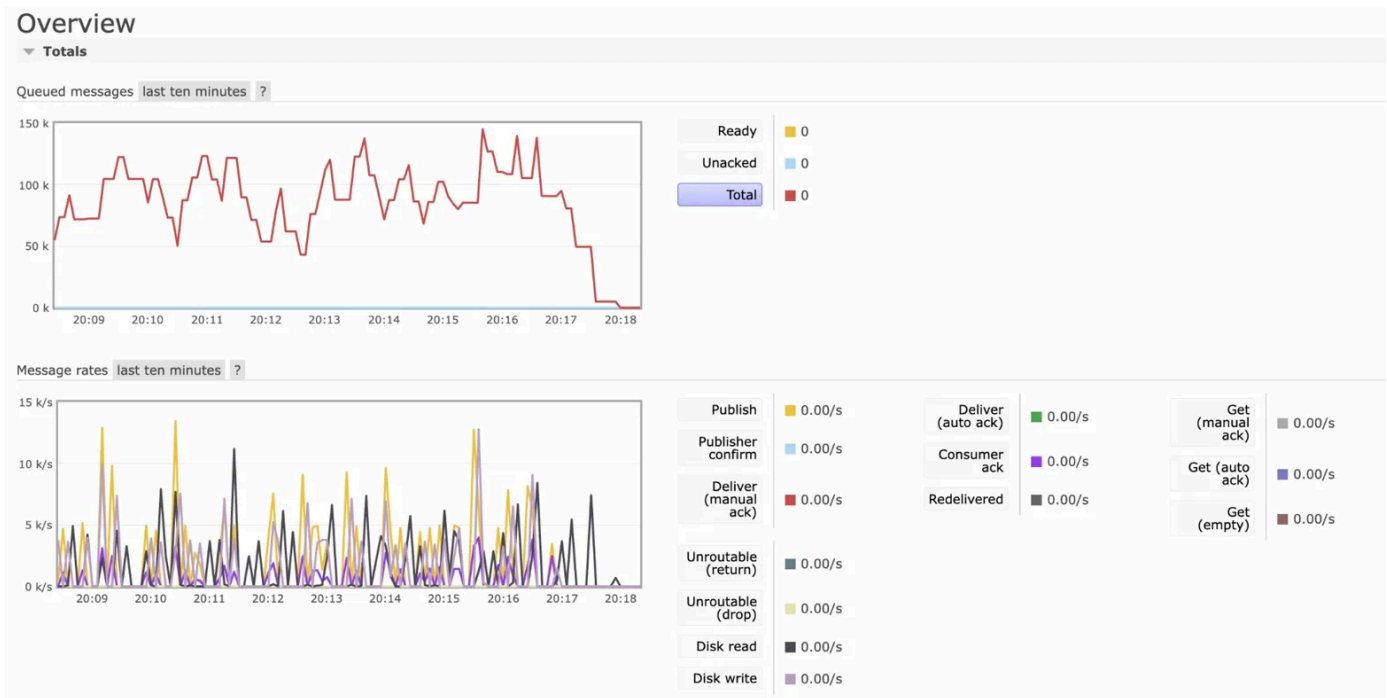
===== RESULTS =====

Messages sent: 9998278
Messages failed: 1722
Total time: 507.72 s
Throughput: 19693 msg/s
Mean latency: 2.3 ms
Median latency: 0 ms
p95 latency: 0 ms
p99 latency: 1 ms
Min/Max latency: 0 / 19979 ms

===== METRICS API RESULTS =====

```
{  
  "totalMessages" : 596312,  
  "totalUsers" : 99929,  
  "totalRooms" : 7,  
  "analytics" : {  
    "totalMessages" : 726589,  
    "messagesPerMinute" : [ {  
      "minute" : "2026-03-27 03:07:00",  
      "count" : 3296  
    }  
  ]  
}
```

Messages sent: 9,998,278
Messages failed: 1,722 (0.017%)
Total time: 507.72s (~8.5 minutes)
Throughput: 19,693 msg/s
Mean latency: 2.3ms
p99 latency: 1ms
DB written: 726,589
DB failed: 0



written time

350186 2026-03-27 03:06:48

written time

429072 2026-03-27 03:09:20

written time

543941 2026-03-27 03:13:02

DB write progression — no

memory leak:

Time	DB Count
t+3:48	350,186
t+6:20	429,072
t+10:02	543,941
Final	726,589

Note: Target of 80% maximum throughput ($\approx 119,000$ msg/s) could not be sustained due to MySQL write bottleneck on t3.micro. Effective end-to-end maximum including persistence is $\sim 20,000$ msg/s. Endurance test ran at 19,693 msg/s = 99% of effective maximum. WebSocket send capacity alone reaches 149,066 msg/s, decoupled from DB writes via write-behind pattern.

4.5 MySQL Buffer Pool Statistics

Metric	Value
Buffer pool hit ratio	99.94%
Buffer pool pages free	0 (fully utilized)
Read requests	105,661,295
Physical disk reads	67,235
Write requests	26,481,951

4.6 Query Optimization

Implemented:

- Table partitioning: `PARTITION BY KEY(room_id) PARTITIONS 20`
- Composite indexes with leftmost prefix rule
- `INSERT IGNORE` for idempotent writes
- Prepared statements via Spring JdbcTemplate `batchUpdate`
- `rewriteBatchedStatements=true` — combines INSERTs into one network round trip

Trade-offs:

- Redis caching would further reduce Query 3 from 292ms, but adds operational complexity. Not implemented due to t3.micro memory constraints.
- Materialized views considered but not implemented; `message_stats` table serves the same purpose.

4.7 Write Performance Summary

Test	Messages	Throughput	p99	DB Written	DB Failed
Baseline (500K)	500K	120,649 msg/s	1ms	~178,592	0
Stress (1M)	1M	20,093 msg/s	1ms	105,923	0
Endurance (10M)	10M	19,693 msg/s	1ms	726,589	0

4.8 Bottleneck Analysis

Primary: MySQL on t3.micro — buffer pool fully utilized, batch writes cause CPU spikes to 87%. Solution: upgrade to RDS t3.medium.

Secondary: Single consumer — handles both WebSocket broadcast and DB writes. Dedicated writer thread pool partially mitigates this.

RabbitMQ queue buildup — peaked at 250k under 1M load. Write-behind pattern successfully decouples real-time delivery from persistence.

5. Configuration Details

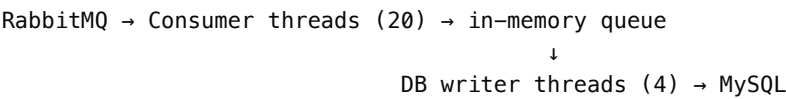
Database

Parameter	Value
Engine	MySQL 8.0, InnoDB
Partitioning	PARTITION BY KEY(room_id) PARTITIONS 20
Message TTL	60,000ms (RabbitMQ)
Idempotency	INSERT IGNORE on (message_id, room_id)

Consumer-v3

Parameter	Value
Consumer threads	20 (one per room)
DB writer threads	4
Batch size	500 messages
Flush interval	500ms
DB connection pool	20 (HikariCP)
Retry attempts	3 with exponential backoff

Thread Pool Design



ALB Settings

Parameter	Value
Listener	HTTP:80
Target port	8080
Stickiness	Load balancer cookie, 1 day
Health check	/health, interval 30s
Idle timeout	4000s

Instance Types

Component	Instance	Count
WS Server	t3.micro	2
Consumer	t3.micro	1
RabbitMQ	t3.micro	1
MySQL	t3.micro	1